Design and Simulate Horizontal Database Sharding using MongoDB

Date: 08.08.2026 Name: Amrit Gopinath
Ex. No: 4 Reg No: 3122 24 5001 015

To design and simulate Horizontal Database Sharding using MongoDB by deploying a sharded cluster with Docker Compose, distributing the Student collection of a College database across multiple shards, and analysing the scalability and query performance of the resulting distributed database system.

Software Used and its Configuration

Software	Role	Purpose in this Assignment
Docker Desktop (WSL2 backend)	Container Runtime	Runs and networks the four MongoDB containers that make up the sharded cluster.
Docker Compose	Orchestration	Declares configsvr1, shard1svr1, shard2svr1 and mongos as one multi-container service and brings them up together on a shared bridge network.
mongo:7.0 (Docker image)	Database Engine	Run once per container as mongod with --configsvr / --shardsvr flags for the config and shard servers, and once as the mongos query router.
mongosh	Database Shell	Bundled inside the image; reached via docker exec to initiate replica sets, add shards, enable sharding and run queries.

System Design and Architecture

Config Server: stores the cluster metadata (shard list and chunk map).

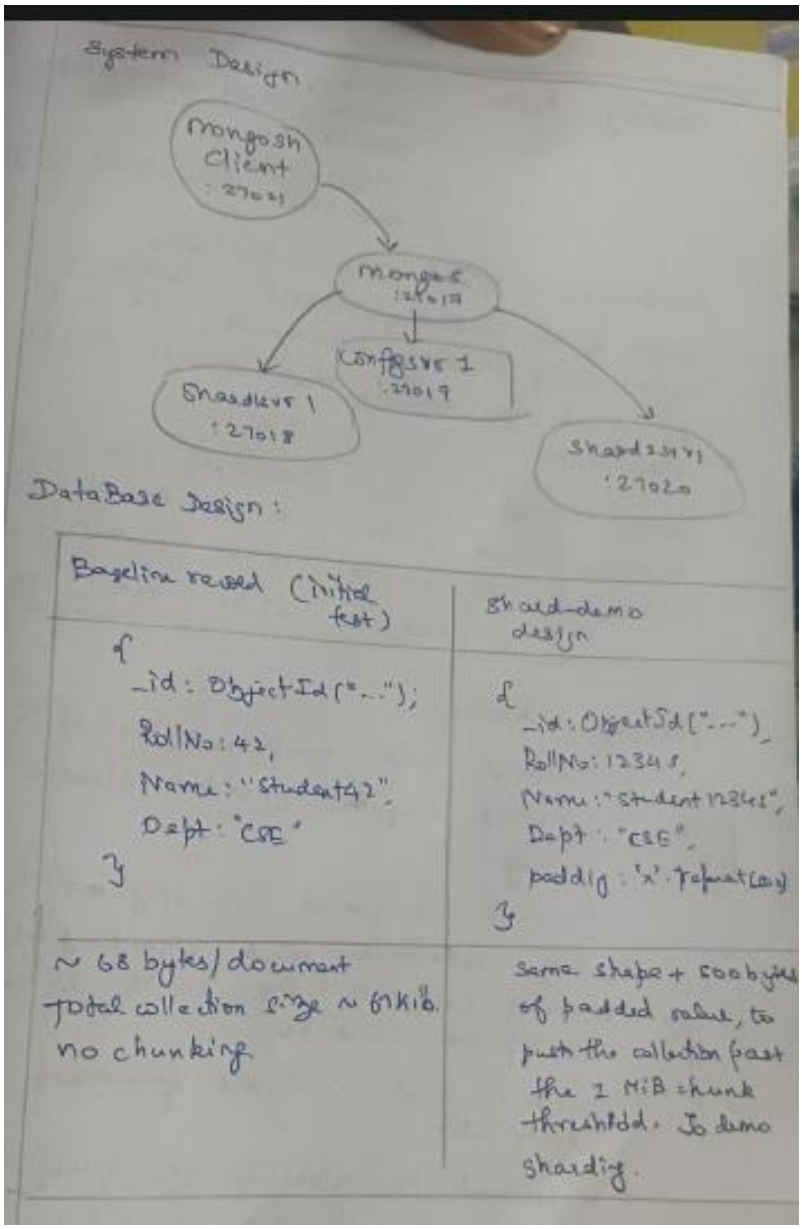
Shard Servers: each holds one range of the Student collection, keyed on RollNo.

mongos Router: the only component the client talks to; it consults the config server and routes each query to the shard that owns the requested RollNo range.*)*

Design and Simulate Horizontal Database Sharding using MongoDB

Date: 08.08.2026
Ex. No: 4

Name: Amrit Gopinath
Reg No: 3122 24 5001 015



Test Cases (Screenshots)

Screenshots captured while executing the cluster setup and sharding commands through mongosh.

Design and Simulate Horizontal Database Sharding using MongoDB

Date: 08.08.2026
Ex. No: 4

Name: Amrit Gopinath
Reg No: 3122 24 5001 015

```
amrit@AMRITG main ≡ ?15 ~5                                                    took 171ms  21:24:36
~\Documents\College\SEM-5\System-Design\mongo-sharding ❯ docker exec -it configsvr1 mongosh --port 27019 --eval "rs.initiate({_id:
'configReplSet', configsvr: true, members: [{_id: 0, host: 'configsvr1:27019'}]})"
{ ok: 1 }

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug configsvr1
    Learn more at https://docs.docker.com/go/debug-cli/
amrit@AMRITG main ≡ ?15 ~5                                                    took 2.419s  21:24:46
~\Documents\College\SEM-5\System-Design\mongo-sharding ❯ docker exec -it shard1svr1 mongosh --port 27018 --eval "rs.initiate({_id:
'shard1ReplSet', members: [{_id: 0, host: 'shard1svr1:27018'}]})"
{ ok: 1 }

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug shard1svr1
    Learn more at https://docs.docker.com/go/debug-cli/
amrit@AMRITG main ≡ ?15 ~5                                                    took 1.544s  21:24:55
~\Documents\College\SEM-5\System-Design\mongo-sharding ❯ docker exec -it shard2svr1 mongosh --port 27020 --eval "rs.initiate({_id:
'shard2ReplSet', members: [{_id: 0, host: 'shard2svr1:27020'}]})"
{ ok: 1 }

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug shard2svr1
    Learn more at https://docs.docker.com/go/debug-cli/
```

```
[direct: mongos] test> sh.addShard("shard1ReplSet/shard1svr1:27018")
{
  shardAdded: 'shard1ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1786204525, i: 6 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1786204525, i: 6 })
}
[direct: mongos] test> sh.addShard("shard2ReplSet/shard2svr1:27020")
{
  shardAdded: 'shard2ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1786204570, i: 14 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1786204570, i: 4 })
}
```

Design and Simulate Horizontal Database Sharding using MongoDB

Date: 08.08.2026
Ex. No: 4

Name: Amrit Gopinath
Reg No: 3122 24 5001 015

```
switched to db College
[direct: mongos] College> sh.enableSharding("College")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1786204599, i: 10 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1786204599, i: 4 })
}
[direct: mongos] College> db.Student.createIndex({ RollNo: 1 })
RollNo_1
[direct: mongos] College> sh.shardCollection("College.Student", { RollNo: 1 })
{
  collectionsharded: 'College.Student',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1786204650, i: 30 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1786204650, i: 30 })
}
[direct: mongos] College> for (let i = 1; i <= 1000; i++) {
|   db.Student.insertOne({ RollNo: i, Name: "Student" + i, Dept: "CSE" })
| }
{
```

```
]
[direct: mongos] College> db.Student.getShardDistribution()
Shard shard2ReplSet at shard2ReplSet/shard2svr1:27020
{
  data: '67KiB',
  docs: 1000,
  chunks: 1,
  'estimated data per chunk': '67KiB',
  'estimated docs per chunk': 1000
}
---
Totals
{
  data: '67KiB',
  docs: 1000,
  chunks: 1,
  'Shard shard2ReplSet': [
    '100 % data',
    '100 % docs in cluster',
    '68B avg obj size on shard'
  ]
}
```

Design and Simulate Horizontal Database Sharding using MongoDB

Date: 08.08.2026
Ex. No: 4

Name: Amrit Gopinath
Reg No: 3122 24 5001 015

The reasoning behind this output is – the sizes of the document is very small for clustering to even happen. Hence I've deleted the documents and added 50k new ones.

```
[direct: mongos] College> db.Student.deleteMany({})
{ acknowledged: true, deletedCount: 1000 }
[direct: mongos] College> db.Student.countDocuments()
0
[direct: mongos] College> use config
switched to db config
[direct: mongos] config> db.settings.updateOne({ _id: "chunksize" }, { $set: { value: 1 } }, { upsert: true })
{
  acknowledged: true,
  insertedId: 'chunksize',
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
[direct: mongos] config> use College
switched to db College
[direct: mongos] College> for (let i = 1001; i <= 50000; i++) {
|   db.Student.insertOne({ RollNo: i, Name: "Student" + i, Dept: "CSE", padding: "x".repeat(500) })
| }
{
  acknowledged: true,
  insertedId: ObjectId('6a78772e09f45dec2dc2c765')
}
[direct: mongos] College> db.Student.getShardDistribution()
```

```
[direct: mongos] College> db.Student.getShardDistribution()
Shard shard1ReplSet at shard1ReplSet/shard1svr1:27018
{
  data: '13MiB',
  docs: 23322,
  chunks: 1,
  'estimated data per chunk': '13MiB',
  'estimated docs per chunk': 23322
}
---
Shard shard2ReplSet at shard2ReplSet/shard2svr1:27020
{
  data: '14.32MiB',
  docs: 25678,
  chunks: 1,
  'estimated data per chunk': '14.32MiB',
  'estimated docs per chunk': 25678
}
---
Totals
{
  data: '27.32MiB',
  docs: 49000,
  chunks: 2,
  'Shard shard1ReplSet': [
    '47.57 % data',
    '47.59 % docs in cluster',
    '584B avg obj size on shard'
  ],
  'Shard shard2ReplSet': [
    '52.42 % data',
    '52.4 % docs in cluster',
    '585B avg obj size on shard'
  ]
}
[direct: mongos] College>
```

Design and Simulate Horizontal Database Sharding using MongoDB

Date: 08.08.2026
Ex. No: 4

Name: Amrit Gopinath
Reg No: 3122 24 5001 015

Performance / Shard Distribution Table

Result of db.Student.getShardDistribution() after loading 49,000 documents with a 1 MiB chunk size, showing the balancer final placement.

Shard	Host	Docs	Data Size	% Docs	% Data
shard1ReplSet	shard1svr1:27018	23,322	13.00 MiB	47.59%	47.57%
shard2ReplSet	shard2svr1:27020	25,678	14.32 MiB	52.40%	52.42%
Total	-	49,000	27.32 MiB	100%	100%

Learning Outcomes

- Deployed and configured a multi-container MongoDB sharded cluster - config server, shard servers, and mongos router - using Docker Compose.
- Enabled horizontal sharding on the College database and sharded the Student collection using RollNo as an appropriate shard key.
- Verified and analysed document distribution across shards using sh.status() and db.Student.getShardDistribution().
- Understood the scalability and query-routing benefits of a sharded MongoDB cluster compared to a standalone MongoDB instance.